

**WYDZIAŁ
ELEKTROTECHNIKI
I INFORMATYKI**
POLITECHNIKI RZESZOWSKIEJ

Arkadiusz Kot

Edytor Logów Geolokalizacyjnych

Praca dyplomowa inżynierska

Opiekun pracy:
dr inż. Paweł Górka

Rzeszów 2026

Spis treści

Spis treści

Wstęp.....	4
1 Charakterystyka porównawcza istniejących na rynku rozwiązań do analizy danych geolokalizacyjnych.....	5
1.1 Strava – sieć społecznościowa dla sportowców.....	5
1.2 Komoot – zaawansowane narzędzie do planowania oraz nawigacji	6
1.3 Garmin Connect - ekosystem analityczny producenta sprzętu	7
1.4 Podsumowanie analizy i uzasadnienie potrzeby stworzenia własnego produktu.	8
2 Architektura i projekt autorskiej aplikacji GeoLog	9
2.1 Założenia projektowe i funkcjonalne	9
2.2 Architektura systemu.....	10
2.3 Projekt bazy danych.	11
2.4 Projekt interfejsu programistycznego aplikacji (API).	13
3 Technologie i implementacje kluczowych etapów	15
3.1 Wykorzystane technologie i środowisko programistyczne.....	15
3.1.1 Technologie warstwy serwerowej (backend).....	16
3.1.2 Technologie warstwy klienckiej (frontend)	18
3.1.3 Środowisko pracy i narzędzia deweloperskie.....	19
Literatura	21

Wstęp

Dynamiczny rozwój technologii geolokalizacyjnych oraz wszechobecność urządzeń wyposażonych w odbiorniki GPS zrewolucjonizowały sposób, w jaki ludzie rejestrują i analizują swoją aktywność fizyczną. W odpowiedzi na te trendy, na rynku oprogramowania powstał szeroki wachlarz zaawansowanych aplikacji internetowych i mobilnych. Wśród nich wyróżnić można kilka dominujących platform, z których każda skupia się na innej grupie docelowej i zestawie funkcjonalności. Aplikacja Strava pozycjonowana jest jako globalna sieć społecznościowa dla sportowców, która umożliwia szczegółową analizę treningów i rywalizację poprzez integrację z urządzeniami monitorującymi [1]. Z kolei platforma Komoot specjalizuje się w zaawansowanym planowaniu tras, oferując funkcjonalność nawigacji oraz importu i eksportu danych, przy mniejszym nacisku na analizę wyników sportowych [2]. Trzecim filarem rynku jest Garmin Connect, stanowiący zamknięty ekosystem dla użytkowników urządzeń marki Garmin, który dostarcza najbardziej dogłębną na rynku analizę parametrów treningowych i danych biometrycznych [3].

Głównym celem niniejszej pracy inżynierskiej było zaprojektowanie i zaimplementowanie kompletnej aplikacji internetowej typu Full-Stack, noszącej nazwę GeoLog, która pełniłaby rolę wszechstronnego edytora i analizatora danych geolokalizacyjnych. W przeciwieństwie do istniejących, często wyspecjalizowanych rozwiązań, celem było stworzenie uniwersalnego narzędzia, które łączyłoby w sobie zarówno możliwość importu istniejących tras, jak i tworzenia nowych od podstaw w intuicyjnym kreatorze wizualnym.

Aby zrealizować główny cel pracy, postawiono następujące cele szczegółowe:

1. Zaprojektowanie i implementacja bezpiecznego systemu rejestracji i uwierzytelniania użytkowników, zabezpieczonego mechanizmem weryfikacji adresów e-mail.
2. Opracowanie modułu do importu tras z plików w standardowym formacie GPX oraz ich eksportu.
3. Implementacja algorytmów do automatycznego obliczania kluczowych statystyk trasy, takich jak dystans, suma przewyższeń i spadków, a w przypadku danych z pliku GPX, również czas trwania i prędkość.
4. Stworzenie interaktywnego kreatora tras, który umożliwi użytkownikowi projektowanie ścieżki poprzez manualne dodawanie punktów na mapie.
5. Zaimplementowanie modułu do zaawansowanej wizualizacji przebiegu trasy na mapie oraz prezentacji jej profilu wysokościowego na dynamicznym wykresie.

6. Umożliwienie użytkownikom zarządzania swoimi trasami, w tym edycji metadanych (nazwa, opis) oraz udostępniania ich publicznie innym członkom społeczności.

Struktura niniejszej pracy została podporządkowana procesowi realizacji postawionych celów. W Rozdziale 1 dokonano szczegółowej charakterystyki porównawczej istniejących na rynku rozwiązań. Rozdział 2 zawiera kompleksowy opis architektury i projektu aplikacji GeoLog. W Rozdziale 3 omówiono wykorzystane technologie oraz przedstawiono implementację kluczowych algorytmów i modułów. Pracę zamyka rozdział zawierający podsumowanie, weryfikację osiągniętych celów oraz wnioski końcowe.

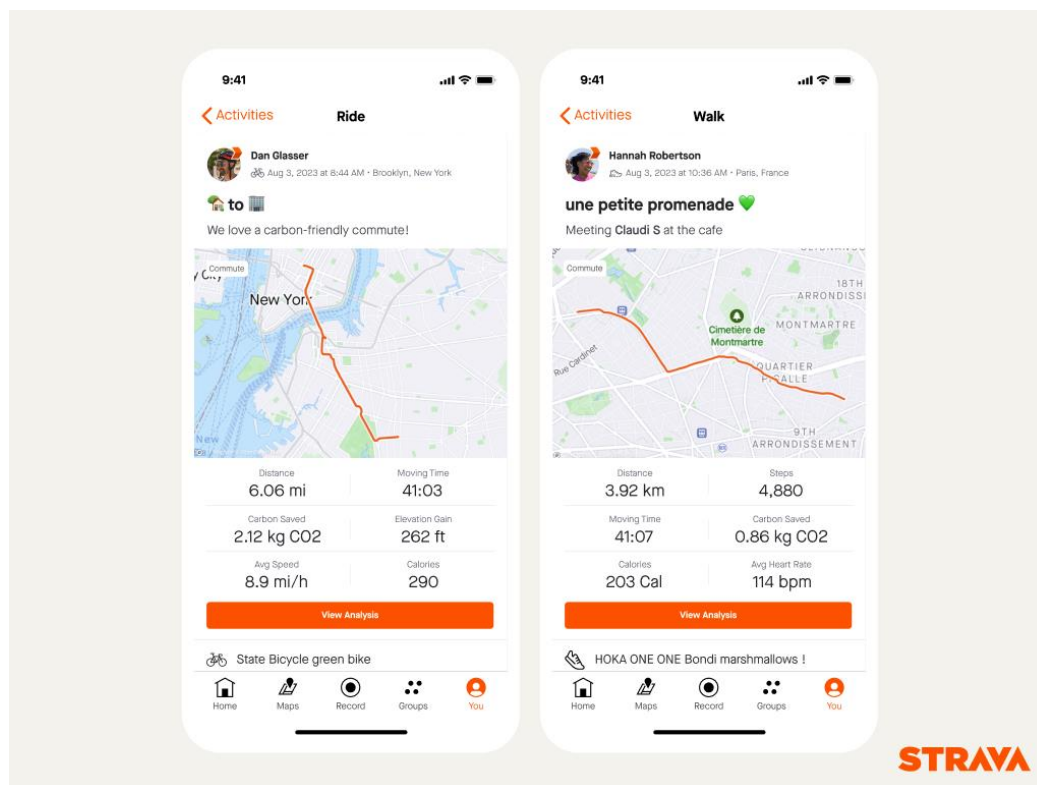
1 Charakterystyka porównawcza istniejących na rynku rozwiązań do analizy danych geolokalizacyjnych

W niniejszym rozdziale szczegółowo opisano aplikacje służące do analizy i edycji danych geolokalizacyjnych. Należą do nich Strava [1], Komoot [2] oraz Garmin Connect [3]

1.1 Strava – sieć społecznościowa dla sportowców

Platforma Strava, stworzona w 2009 roku przez Marka Gaineya i Michaela Horvatha, jest aplikacją internetową i mobilną służącą do monitorowania aktywności fizycznej, z silnym naciskiem na aspekty społecznościowe [4]. Kluczową funkcjonalnością aplikacji jest możliwość precyzyjnego śledzenia i analizy treningów, obejmująca takie parametry jak pokonany dystans, czas, prędkość, a także sumę przewyższeń i kadencję. Zebrane dane są następnie wizualizowane w formie czytelnych statystyk, interaktywnych map oraz wykresów profilu trasy (rys. 1.1).

Rysunek 1.1. Interfejs aplikacji Strava z analizą aktywności

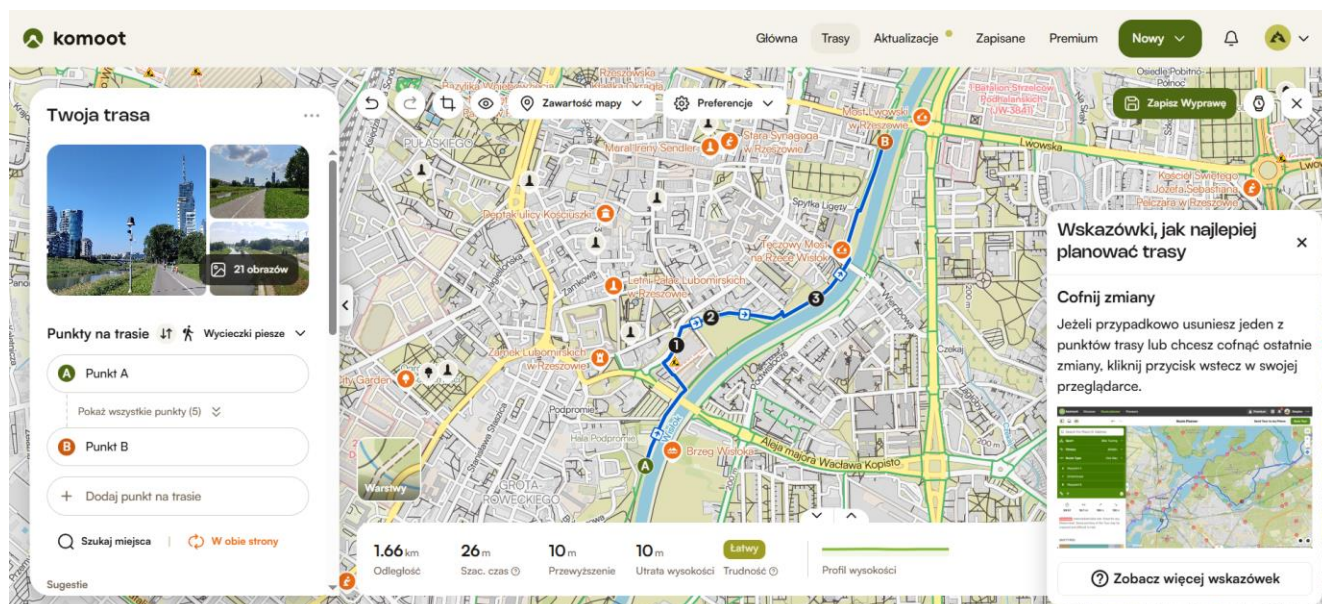


Istotnym elementem, który wyróżnia Stravę na tle konkurencji, jest rozbudowany moduł społecznościowy. Umożliwia on użytkownikom nie tylko dodawanie znajomych i publiczne udostępnianie zarejestrowanych aktywności, ale również komentowanie i przyznawanie wirtualnych wyróżnień (kudos). Platforma wprowadza także silne elementy grywalizacji, z których najbardziej znaną są Segmenty – predefiniowane odcinki dróg lub szlaków, na których użytkownicy mogą automatycznie rywalizować o jak najlepszy czas. Platforma wspiera ponad 30 różnych dyscyplin sportowych, co czyni ją uniwersalnym narzędziem dla szerokiego grona sportowców. Pełny dostęp do zaawansowanych funkcji, takich jak szczegółowe planowanie treningów, analiza mocy czy funkcja bezpieczeństwa Beacon (udostępnianie lokalizacji w czasie rzeczywistym), wymaga jednak aktywnej subskrypcji, co może stanowić barierę dla części użytkowników [5].

1.2 Komoot – zaawansowane narzędzie do planowania oraz nawigacji

Aplikacja Komoot, wprowadzona na rynek w 2013 roku, została zaprojektowana jako specjalistyczne narzędzie do planowania i nawigacji podczas aktywności na świeżym powietrzu, takich jak turystyka piesza, kolarstwo górskie i szosowe oraz bieganie [6]. Głównym wyróżnikiem platformy jest zaawansowany planer tras, który umożliwia użytkownikom precyzyjne wyznaczanie przebiegu wycieczki "od punktu do punktu". Algorytmy planowania, określane przez twórców jako "planowanie pod kątem przygody", optymalizują trasę nie tylko pod kątem najkrótszej drogi, ale również z uwzględnieniem specyfiki wybranej dyscypliny, profilu wysokościowego oraz, co jest unikalną cechą, rodzaju nawierzchni (asfalt, szuter, ścieżki leśne). Interfejs planera (rys. 1.2) pozwala na dynamiczne modyfikowanie trasy poprzez przeciąganie punktów oraz dodawanie tzw. "Atrakcji" (ang. Highlights) – interesujących miejsc polecanych przez społeczność.

Rysunek 1.2. Interfejs planera tras w aplikacji Komoot



Oprócz funkcji planistycznych, Komoot oferuje również standardowe możliwości rejestrowania aktywności, monitorowania bieżącej prędkości i wysokości oraz importu i eksportu plików w formacie GPX. Aplikacja mobilna udostępnia funkcję nawigacji głosowej turn-by-turn oraz możliwość pobierania map do użytku w trybie offline. Model

biznesowy Komoot różni się od subskrypcyjnego – opiera się on na jednorazowych opłatach za odblokowanie dostępu do map offline dla poszczególnych regionów lub całego świata. Choć Komoot posiada elementy społecznościowe, takie jak udostępnianie zrealizowanych tras, jego główny nacisk położony jest na funkcjonalność eksploracyjną i nawigacyjną, a nie na rywalizację i analizę sportową [7].

1.3 Garmin Connect - ekosystem analityczny producenta sprzętu

Platforma Garmin Connect stanowi integralną część ekosystemu urządzeń firmy Garmin, pełniąc funkcję centralnego repozytorium do gromadzenia i zaawansowanej analizy danych. W odróżnieniu od platform uniwersalnych, jej pełne wykorzystanie jest ściśle powiązane z posiadaniem dedykowanego sprzętu, takiego jak zegarki sportowe z serii Forerunner czy Fenix, lub liczników rowerowych z serii Edge. Główną siłą Garmin Connect jest głębia analizy danych, która wykracza poza podstawowe statystyki trasy. System przetwarza i prezentuje szeroki zakres wskaźników fizjologicznych i wydajnościowych, takich jak pułap tlenowy (VO2 max), próg mleczanowy (dla biegania i kolarstwa), obciążenie treningowe, czas regeneracji, a także całodobowe dane biometryczne, jak jakość snu, poziom stresu czy Body Battery (rys. 1.3).

Rysunek 1.3. Wybrane analizy w aplikacji Garmin Connect



Aplikacja udostępnia również narzędzia do planowania tras oraz moduł społecznościowy, pozwalający na dzielenie się aktywnościami z innymi użytkownikami platformy. Funkcjonalności te mają jednak charakter uzupełniający i często są postrzegane jako mniej intuicyjne i rozbudowane w porównaniu do wyspecjalizowanych liderów rynku, takich jak Komoot czy Strava. Wartość platformy leży przede wszystkim w kompleksowej integracji ze sprzętem i możliwości długoterminowego monitorowania postępów treningowych oraz ogólnego stanu zdrowia użytkownika w ramach jednego, spójnego ekosystemu [8].

1.4 Podsumowanie analizy i uzasadnienie potrzeby stworzenia własnego produktu.

Przeprowadzona w niniejszym rozdziale analiza porównawcza wiodących na rynku aplikacji do zarządzania danymi geolokalizacyjnymi pozwala na zidentyfikowanie wyraźnej specjalizacji poszczególnych platform. Z jednej strony rynku znajduje się Strava, która zdominowała segment społecznościowo-sportowy, budując swoją wartość na mechanizmach rywalizacji i interakcji między użytkownikami. Z drugiej strony, Komoot pozycjonuje się jako niezastąpione narzędzie do precyzyjnego, zautomatyzowanego planowania tras w nieznanym terenie, skupiając się na aspekcie eksploracyjnym. Trzeci model reprezentuje Garmin Connect, oferujący najgłębszą na rynku analizę danych fizjologicznych, jednak jego funkcjonalność jest nierozdzielnie związana z zamkniętym ekosystemem sprzętowym jednego producenta. W celu syntetycznego zobrazowania tych różnic, kluczowe funkcjonalności analizowanych aplikacji zestawiono w tabeli 1.1.

Tabela 1.1. Zestawienie funkcjonalności analizowanych aplikacji

Funkcjonalność	Strava	Komoot	Garmin Conect
Import i analiza plików GPX	+	+	+
Zaawansowana analiza statystyk	++	+	+++
Wizualizacja trasy na mapie	+	++	+
Zaawansowane planowanie trasy	+	+++	+
Nawigacja turn-by-turn	+	++	+
Aspekt społecznościowy	+++	+	--
Zależność od ekosystemu	Nie	Nie	Tak

Legenda: +++ - funkcja kluczowa, ++ - funkcja rozbudowana, + - funkcja obecna, -- - brak funkcji

Szczegółowa analiza wykazała, że choć każda z platform oferuje możliwość importu plików GPX, funkcja ta jest często traktowana jako dodatek do głównego modelu biznesowego – czy to subskrypcji, sprzedaży sprzętu, czy opłat za dostęp do map. Na rynku zidentyfikowano niszę dla rozwiązania, które byłoby w pełni skoncentrowane na jednej, kluczowej potrzebie: dostarczeniu prostego, lecz potężnego narzędzia do zarządzania, analizy i wizualizacji własnych plików GPX, bez konieczności angażowania

się w rozbudowane funkcje społecznościowe czy zamknięte ekosystemy sprzętowe. Istniejące platformy często wymagają płatnej subskrypcji dla dostępu do zaawansowanych analiz lub narzucają użytkownikowi określony sposób interakcji, co może być barierą dla entuzjastów danych i osób ceniących sobie pełną kontrolę nad swoimi informacjami.

Zidentyfikowana nisza rynkowa stała się podstawą do sformułowania założeń dla autorskiego projektu aplikacji GeoLog. Głównym celem stało się stworzenie lekkiej, wydajnej i niezależnej aplikacji internetowej, która da użytkownikowi pełną kontrolę nad jego danymi geolokalizacyjnymi. Założono, że aplikacja powinna oferować nie tylko zaawansowane możliwości wizualizacji i analizy zaimportowanych tras, ale także udostępnić intuicyjny kreator do manualnego projektowania nowych ścieżek od podstaw. Takie połączenie funkcjonalności, przy jednoczesnym zachowaniu prostoty obsługi i braku barier finansowych, stanowi o unikalnej propozycji wartości projektu i uzasadnia potrzebę jego realizacji.

2 Architektura i projekt autorskiej aplikacji GeoLog

2.1 Założenia projektowe i funkcjonalne

Proces projektowy aplikacji GeoLog oparto na zbiorze precyzyjnie zdefiniowanych założeń, które podzielono na wymagania funkcjonalne i niefunkcjonalne. Celem było stworzenie kompletnego, niezawodnego i intuicyjnego w obsłudze systemu do zarządzania danymi geolokalizacyjnymi. Poniższe akapity szczegółowo opisują kluczowe wymagania, które zostały zrealizowane w ramach projektu.

Pierwszą grupę stanowią wymagania funkcjonalne dotyczące zarządzania użytkownikami. Zaimplementowany system musiał zapewniać możliwość bezpiecznej rejestracji nowego użytkownika za pomocą adresu e-mail, unikalnej nazwy użytkownika i hasła. W celu zapewnienia autentyczności podawanych danych, proces ten został zabezpieczony poprzez mechanizm weryfikacji adresu e-mail, polegający na wysłaniu wiadomości z unikalnym linkiem aktywacyjnym. Po pomyślnej weryfikacji, użytkownik uzyskiwał możliwość zalogowania się do systemu. W ramach zarządzania kontem, system musiał również umożliwiać zalogowanemu użytkownikowi zmianę hasła, warunkowaną podaniem bieżącego hasła, oraz personalizację profilu poprzez wgranie własnego awatara w formacie graficznym.

Druga, kluczowa grupa wymagań dotyczyła funkcjonalności związanych z zarządzaniem trasami. Aplikacja musiała oferować dwa sposoby tworzenia tras. Pierwszy z nich to możliwość przesłania przez zalogowanego użytkownika istniejącego pliku w formacie GPX, wraz z metadanymi takimi jak nazwa, opis oraz poziom widoczności (prywatna, niepubliczna, publiczna). System został zobligowany do automatycznego przetworzenia takiego pliku, w tym do ekstrakcji sekwencji punktów (współrzędnych, opcjonalnej wysokości i znacznika czasu). Drugim sposobem tworzenia trasy jest interaktywny kreator, który umożliwia użytkownikowi manualne projektowanie nowej ścieżki poprzez wskazywanie kolejnych punktów na mapie.

Niezależnie od sposobu utworzenia trasy, aplikacja miała za zadanie automatycznie obliczać i zapisywać jej podstawowe statystyki, w tym całkowity dystans, sumę przewyższeń i spadków, a dla tras importowanych z plików GPX – również łączny czas trwania oraz prędkość średnią i maksymalną. Zrealizowano również wymaganie dotyczące udostępniania szczegółowego widoku trasy, który prezentuje jej przebieg na interaktywnej

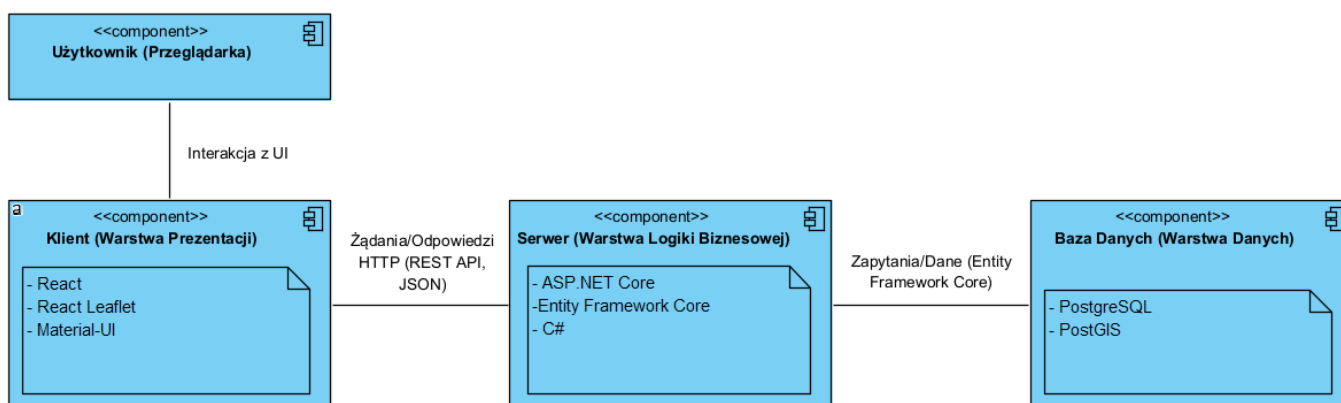
mapie oraz wizualizuje jej profil wysokościowy lub prędkości na dynamicznych wykresach. Użytkownik będący właścicielem trasy musiał otrzymać możliwość edycji jej nazwy i opisu, a także jej trwałego usunięcia. System miał również udostępniać publiczny panel ("Odkrywaj"), na którym prezentowane są wszystkie trasy oznaczone jako publiczne, z funkcjonalnością wyszukiwania i sortowania.

Oprócz wymagań funkcjonalnych, zdefiniowano również wymagania niefunkcjonalne, które określają jakość działania systemu. Do najważniejszych z nich należały: wydajność, rozumiana jako szybki czas odpowiedzi serwera na żądania, zwłaszcza podczas przetwarzania dużych plików GPX; bezpieczeństwo, zapewnione przez hashowanie haseł i autoryzację opartą na tokenach JWT; oraz użyteczność, osiągnięta dzięki zastosowaniu intuicyjnego i responsywnego interfejsu użytkownika, który dostosowuje się do różnych rozmiarów ekranu.

2.2 Architektura systemu.

Projektując architekturę oprogramowania, dąży się do stworzenia struktury, która nie tylko spełnia wymagania funkcjonalne, ale również zapewnia pożądane atrybuty jakościowe, takie jak wydajność, niezawodność i łatwość w modyfikacji [9]. W przypadku aplikacji GeoLog, której zadaniem jest przetwarzanie i wizualizacja danych geoprzestrzennych w środowisku internetowym, zdecydowano się na implementację sprawdzonego wzorca architektonicznego Klient-Serwer w jego klasycznym, trójwarstwowym wariancie. Jak argumentuje Martin Fowler, separacja warstwy prezentacji, logiki biznesowej oraz warstwy dostępu do danych jest fundamentalną zasadą pozwalającą na budowę elastycznych i skalowalnych systemów [10]. Taki podział umożliwia niezależny rozwój i wdrażanie poszczególnych komponentów aplikacji, co jest kluczowe w nowoczesnych procesach inżynierii oprogramowania. Schemat przyjętej architektury przedstawiono na rysunku 2.1.

Rys. 2.1. Ogólny schemat architektury aplikacji GeoLog



Pierwszym komponentem systemu jest warstwa prezentacji (klient), zrealizowana w modelu Aplikacji Jednostronicowej (ang. Single-Page Application, SPA). W przeciwieństwie do tradycyjnych aplikacji wielostronicowych, gdzie każda interakcja użytkownika może skutkować przeładowaniem całej strony, model SPA ładuje początkowy zasób HTML tylko raz, a następnie dynamicznie aktualizuje widoki przy użyciu języka JavaScript [11]. Takie podejście, zaimplementowane w projekcie przy użyciu biblioteki

React, zapewnia użytkownikowi płynne i responsywne doświadczenie, zbliżone do aplikacji desktopowych. Warstwa ta jest w całości odpowiedzialna za renderowanie interfejsu użytkownika, w tym interaktywnych map i wykresów, oraz za całą komunikację z serwerem.

Centralnym elementem systemu jest warstwa logiki biznesowej (serwer), zaimplementowana w technologii ASP.NET Core. Jej głównym zadaniem jest udostępnienie usług poprzez interfejs programistyczny aplikacji (API), zaprojektowany zgodnie ze stylem architektonicznym REST (Representational State Transfer). Zgodnie z pryncypiami REST, zdefiniowanymi w dysertacji Roya Fieldinga, komunikacja opiera się na bezstanowej wymianie komunikatów i operacjach na zasobach identyfikowanych przez unikalne adresy URI [12]. Warstwa serwerowa jest w pełni odpowiedzialna za realizację całej logiki biznesowej: od uwierzytelniania i autoryzacji użytkowników, poprzez walidację danych wejściowych, aż po skomplikowane operacje parsowania plików GPX i agregacji statystyk trasy.

Fundamentem architektury jest warstwa danych, zrealizowana przy użyciu obiektowo-relacyjnego systemu zarządzania bazą danych (SZBD) PostgreSQL. Wybór ten podyktowany był jego zgodnością ze standardem SQL, wysoką wydajnością oraz, co kluczowe dla tego projektu, dostępnością rozszerzenia PostGIS. PostGIS przekształca PostgreSQL w potężną geobazę danych, dostarczając typy danych i funkcje przestrzenne zgodne ze specyfikacjami Open Geospatial Consortium (OGC), co umożliwia efektywne przechowywanie i wykonywanie zapytań na danych geograficznych [13].

Komunikacja pomiędzy warstwą klienta a serwera odbywa się za pomocą protokołu HTTP, a dane przesyłane są w formacie JSON (JavaScript Object Notation), który ze względu na swoją prostotę i czytelność stał się de facto standardem w nowoczesnych aplikacjach webowych. W celu zabezpieczenia dostępu do zasobów API zaimplementowano mechanizm autoryzacji oparty na standardzie JSON Web Tokens (JWT), opisanym w dokumencie RFC 7519 [14]. Po pomyślnym uwierzytelnieniu, serwer wystawia cyfrowo podpisany token, który klient dołącza do kolejnych żądań, co pozwala serwerowi na weryfikację tożsamości użytkownika bez konieczności odwoływania się do bazy danych przy każdym zapytaniu.

2.3 Projekt bazy danych.

Warstwa danych stanowi fundament każdej aplikacji opartej na danych, a jej prawidłowe zaprojektowanie ma kluczowy wpływ na integralność, wydajność i skalowalność całego systemu. W projekcie aplikacji GeoLog zastosowano relacyjny model danych, który, jak opisuje Thomas Connolly w pracy „Database Systems: A Practical Approach to Design, Implementation, and Management”, zapewnia rygorystyczną strukturę i spójność danych poprzez wykorzystanie mechanizmów kluczy głównych, obcych oraz więzów integralności [15]. Proces projektowania logicznego oparto na modelu encji-związków (ang. Entity-Relationship Model), co pozwoliło na formalne zidentyfikowanie podstawowych obiektów (encji), ich właściwości (atrybutów) oraz wzajemnych powiązań (relacji) [16].

Logiczny schemat bazy danych został następnie zaimplementowany w systemie zarządzania bazą danych (SZBD) PostgreSQL, który jest ceniony za swoją niezawodność, zgodność ze standardem ACID (Atomicity, Consistency, Isolation, Durability) oraz zaawansowane możliwości rozszerzania funkcjonalności [17]. Kluczowym elementem implementacji było wykorzystanie rozszerzeń postgis oraz uuid-oss. Pierwsze z nich wprowadza obsługę typów i funkcji geoprzestrzennych, zgodnych ze standardem Simple

Feature Access zdefiniowanym przez Open Geospatial Consortium (OGC), co jest fundamentalne dla głównego celu aplikacji [13]. Drugie natomiast dostarcza funkcje do generowania uniwersalnie unikalnych identyfikatorów (UUID), które zastosowano jako klucze główne w głównych tabelach. Szczegółową strukturę bazy danych przedstawiono na diagramie encji-związków (rys. 2.2).

Rys 2.1 Diagram ERD bazy danych

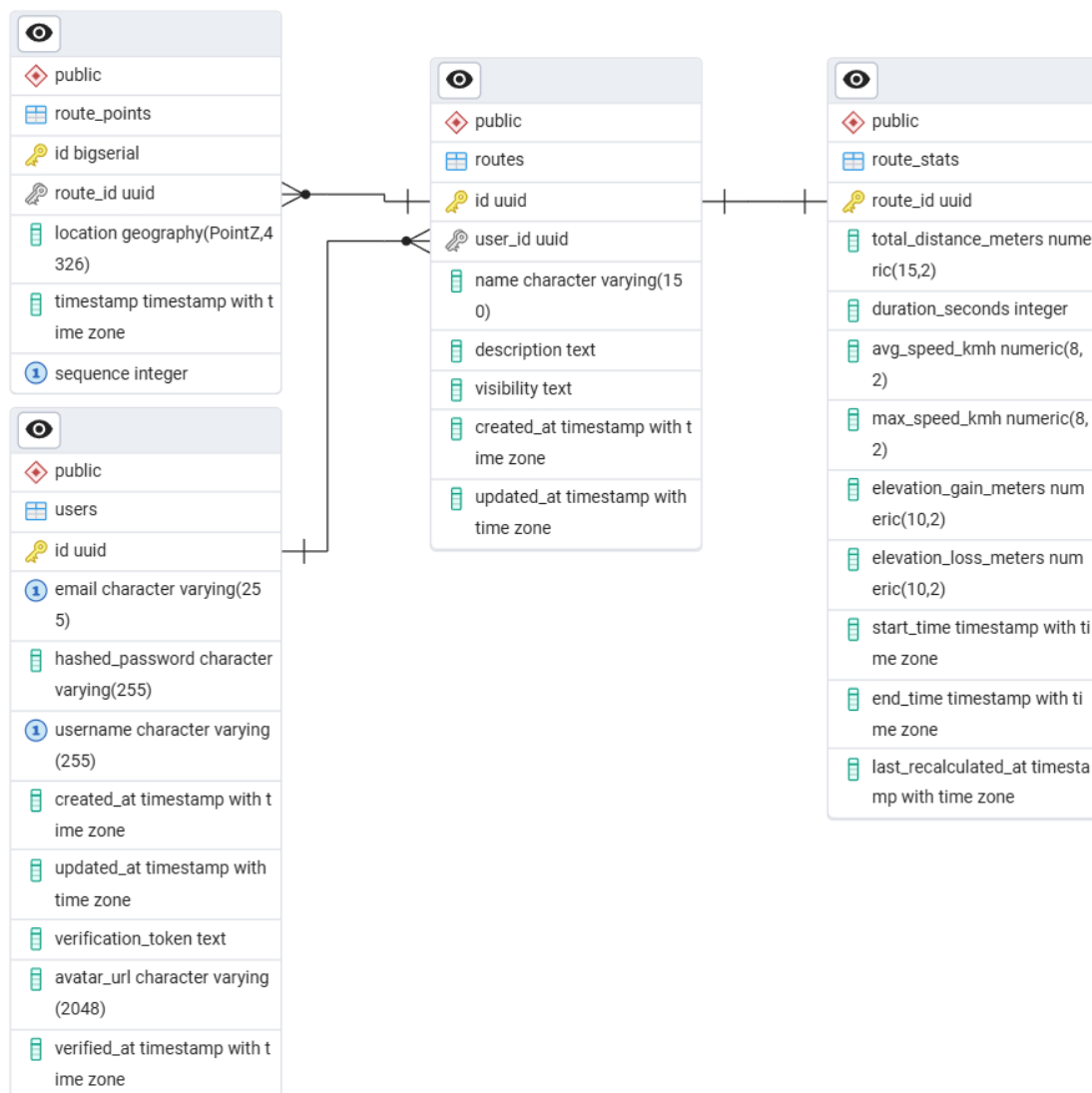


Tabela users jest odpowiedzialna za przechowywanie informacji o kontach użytkowników. Każdy rekord jest jednoznacznie identyfikowany przez klucz główny id typu uuid, którego wartość jest automatycznie generowana przy tworzeniu nowego rekordu. W tabeli znajdują się dane niezbędne do procesu uwierzytelniania, takie jak email oraz username, dla których zdefiniowano więzy unikalności (UNIQUE), aby zapobiec duplikacji kont. Kolumna hashed_password przechowuje skrót hasła użytkownika, wygenerowany jednostronną funkcją kryptograficzną BCrypt, co jest zgodne z najlepszymi praktykami bezpieczeństwa w zakresie przechowywania danych uwierzytelniających [18]. Tabela

zawiera również pola wspierające proces weryfikacji konta (`verification_token`, `verified_at`) oraz personalizację profilu (`avatar_url`).

Tabela `routes` to centralna encja aplikacji, agregująca metadane każdej zapisanej trasy. Tabela połączona jest z tabelą `users` relacją jeden-do-wielu za pośrednictwem klucza obcego `user_id`, co gwarantuje, że każda trasa musi być przypisana do istniejącego użytkownika. Integralność referencyjna jest wzmocniona przez zastosowanie klauzuli `ON DELETE CASCADE`, co oznacza, że usunięcie rekordu użytkownika automatycznie spowoduje usunięcie wszystkich powiązanych z nim tras. W tabeli znajdują się takie atrybuty jak `name` (nazwa trasy), `description` (opis) oraz `visibility` (poziom widoczności), który jest kluczowy dla logiki autoryzacji dostępu do zasobu.

Tabela `route_points` zawiera sekwencję punktów geograficznych dla każdej trasy i jest połączona z tabelą `routes` relacją jeden-do-wielu poprzez klucz obcy `route_id`, również z opcją kaskadowego usuwania. Kluczowym atrybutem jest kolumna `location`, zdefiniowana jako typ `geography(PointZ, 4326)`. Wybór typu `GEOGRAPHY` był świadomą decyzją projektową, podyktowaną potrzebą wykonywania precyzyjnych obliczeń odległości na sferoidzie ziemskiej [13]. Komponent `Z` umożliwia przechowywanie wysokości n.p.m., a identyfikator `SRID 4326` jednoznacznie określa system odniesienia przestrzennego jako `WGS 84`. W celu optymalizacji zapytań przestrzennych, na tej kolumnie został założony specjalistyczny indeks typu `GiST` (`Generalized Search Tree`). Integralność sekwencji punktów w ramach jednej trasy jest zapewniona przez więzy unikalności na parze kolumn (`route_id`, `sequence`).

Ostatnia z głównych tabel, `route_stats`, przechowuje zagregowane statystyki dla każdej trasy, takie jak całkowity dystans, czas trwania czy suma przewyższeń. Jest ona połączona z tabelą `routes` relacją jeden-do-jednego, gdzie klucz główny `route_id` jest jednocześnie kluczem obcym. Wyodrębnienie tych danych do osobnej tabeli jest formą celowej denormalizacji. Taka struktura pozwala na niezwykle szybkie pobieranie zagregowanych statystyk bez konieczności ponownego, kosztownego obliczania ich na podstawie potencjalnie tysięcy punktów przy każdym żądaniu, co znacząco poprawia wydajność aplikacji, zwłaszcza w widokach listujących wiele tras.

2.4 Projekt interfejsu programistycznego aplikacji (API).

Komunikacja pomiędzy oddzielonymi od siebie warstwami klienta i serwera odbywa się za pośrednictwem precyzyjnie zdefiniowanego kontraktu, którym jest interfejs programistyczny aplikacji (API). W projekcie `GeoLog` został on zaprojektowany zgodnie z architektonicznym stylem `REST` (`Representational State Transfer`), który stał się de facto standardem w budowie nowoczesnych systemów rozproszonych. Zgodnie z pryncypiami `REST`, zdefiniowanymi przez Roya Fieldinga, interfejs jest zorientowany na zasoby (w tym przypadku np. użytkownicy, trasy), a nie na zdalne wywoływanie procedur [12]. Każdy zasób posiada unikalny identyfikator (`URI`), a operacje na nim wykonywane są za pomocą standardowych i semantycznie jednoznacznych metod protokołu `HTTP` (`GET`, `POST`, `PUT`, `DELETE`). Takie podejście, w połączeniu z zasadą bezstanowości serwera (każde żądanie od klienta musi zawierać wszystkie informacje niezbędne do jego przetworzenia), zapewnia jednolity i przewidywalny sposób interakcji, co znacząco ułatwia rozwój i integrację komponentów systemu.

Wszystkie dane wymieniane pomiędzy klientem a serwerem są serializowane do formatu `JSON` (`JavaScript Object Notation`). Wybór ten był podyktowany jego lekkością, czytelnością dla człowieka oraz natywnym wsparciem w środowiskach `JavaScript`, co

eliminuje potrzebę skomplikowanego parsowania po stronie klienta. Struktura przesyłanych danych została zdefiniowana za pomocą obiektów transferu danych (DTO), co pozwoliło na oddzielenie wewnętrznego modelu domeny aplikacji od kontraktu publicznego API.

Interfejs API został logicznie podzielony na dwa główne kontrolery, które grupują operacje związane z poszczególnymi zasobami: UsersController do zarządzania użytkownikami i uwierzytelnianiem oraz RoutesController do obsługi wszystkich operacji na trasach. Dostęp do części zasobów jest publiczny (anonimowy), na przykład w celu przeglądania tras publicznych. Jednakże operacje modyfikujące dane (np. tworzenie, edycja, usuwanie) lub dotyczące informacji prywatnych są chronione i wymagają od klienta przedstawienia ważnego tokena JWT (JSON Web Token) w nagłówku Authorization każdego żądania HTTP. Mechanizm ten, opisany w dokumencie RFC 7519, pozwala na bezpieczną i bezstanową autoryzację, gdzie serwer po jednorazowym uwierzytelnieniu użytkownika nie musi przechowywać informacji o jego sesji [14]. Szczegółowy wykaz głównych punktów końcowych (ang. endpoints) API, wraz z ich metodami, opisem i wymogami autoryzacji, przedstawiono w tabelach 2.1 i 2.2.

Tabela 2.1. Główne punkty końcowe API dla zasobu Users

Model HTTP	Ścieżka URL	Opis działania	Wymaga autoryzacji
POST	/api/users/register	Rejestracja nowego użytkownika	Nie
POST	/api/users/login	Uwierzytelnienie użytkownika i zwrot tokena JWT.	Nie
POST	/api/users/verify-email	Weryfikacja adresu e-mail na podstawie tokena.	Nie
GET	/api/users/GetUserId	Pobranie danych zalogowanego użytkownika.	Tak
POST	/api/users/avatar	Przesłanie i aktualizacja awatara użytkownika.	Tak
POST	/api/users/change-password	Zmiana hasła zalogowanego użytkownika.	Tak

Tabela 2.2. Główne punkty końcowe API dla zasobu Routes

Metoda HTTP	Ścieżka URI	Opis działania	Wymaga autoryzacji
POST	/api/routes/upload	Przesłanie i przetworzenie trasy z pliku GPX.	Tak
POST	/api/routes/create	Utworzenie nowej trasy na podstawie punktów z kreatora.	Tak
GET	/api/routes	Pobranie listy tras należących do zalogowanego użytkownika.	Tak
GET	/api/routes/public	Pobranie listy wszystkich tras publicznych, z opcją wyszukiwania i sortowania.	Nie
GET	/api/routes/{id}	Pobranie szczegółowych danych pojedynczej trasy.	Zależnie od widoczności
GET	/api/routes/{id}/points	Pobranie punktów geograficznych dla pojedynczej trasy.	Zależnie od widoczności
PUT	/api/routes/{id}	Aktualizacja metadanych (nazwa, opis) trasy.	Tak (tylko właściciel)
DELETE	/api/routes/{id}	Usunięcie trasy.	Tak (tylko właściciel)

3 Technologie i implementacje kluczowych etapów

3.1 Wykorzystane technologie i środowisko programistyczne

Dobór odpowiedniego stosu technologicznego jest jednym z kluczowych czynników decydujących o sukcesie projektu informatycznego. Wpływa on nie tylko na proces implementacji, ale również na przyszłe możliwości rozwoju, skalowalność i łatwość utrzymania systemu. W ramach realizacji aplikacji GeoLog postawiono na zbiór nowoczesnych, sprawdzonych i dobrze udokumentowanych technologii typu open-source, które tworzą spójny i wydajny ekosystem do budowy aplikacji internetowych. W niniejszym podrozdziale przedstawiono szczegółową charakterystykę wykorzystanych narzędzi, bibliotek i frameworków, wraz z uzasadnieniem ich wyboru w kontekście wymagań projektowych.

3.1.1 Technologie warstwy serwerowej (backend)

Aplikacja serwerowa, stanowiąca serce systemu, została zaimplementowana z wykorzystaniem ekosystemu firmy Microsoft.

Platforma .NET 8 i język C#: Jako fundament wykonawczy aplikacji serwerowej wybrano platformę .NET 8, która jest kompletnym, otwartoźródłowym ekosystemem deweloperskim firmy Microsoft. Wybór ten podyktowany był jej kluczowymi atrybutami, takimi jak wysoka wydajność, bezpieczeństwo i wieloplatformowość, umożliwiające uruchomienie aplikacji na systemach Windows, Linux oraz macOS bez modyfikacji kodu źródłowego. Wydajność platformy .NET wynika z zastosowania zaawansowanego kompilatora JIT (Just-In-Time), który w czasie rzeczywistym tłumaczy kod pośredni (Intermediate Language) na zoptymalizowany kod maszynowy, oraz z wydajnego mechanizmu automatycznego zarządzania pamięcią, znanego, jako Garbage Collector (GC) [19]. Platforma dostarcza również bogaty zbiór bibliotek bazowych (Base Class Library), które oferują gotowe implementacje dla szerokiego spektrum zadań, od operacji na plikach po obsługę protokołów sieciowych.

Język C# w wersji 12, użyty do implementacji całej logiki biznesowej, jest nowoczesnym, obiektowym językiem programowania o statycznym i silnym typowaniu. Ta ostatnia cecha jest niezwykle istotna z perspektywy niezawodności oprogramowania, ponieważ wymusza weryfikację poprawności typów już na etapie kompilacji, co pozwala na wyeliminowanie całej klasy błędów, które w językach o typowaniu dynamicznym mogłyby ujawnić się dopiero w środowisku produkcyjnym [20]. Szczególnie istotną cechą języka, kluczową dla zaimplementowanego systemu, jest wbudowany model programowania asynchronicznego, oparty na słowach kluczowych `async` i `await`. Pozwala on na implementację nieblokujących operacji wejścia-wyjścia (I/O), takich jak zapytania do bazy danych czy odczyt plików. W kontekście aplikacji serwerowej, mechanizm ten zwalnia wątek obsługujący żądanie na czas oczekiwania na zakończenie operacji I/O, umożliwiając mu w tym czasie obsługę innych, przychodzących zapytań. Taki model zarządzania zasobami jest fundamentalny dla osiągnięcia wysokiej skalowalności i responsywności systemu pod dużym obciążeniem [21].

Framework ASP.NET Core: Do budowy interfejsu API wykorzystano framework ASP.NET Core, który cechuje się lekką i modułową architekturą. Jego centralnym elementem jest potok przetwarzania żądań (ang. **middleware pipeline**), pozwalający na sekwencyjną obsługę przychodzących zapytań HTTP przez konfigurowalne komponenty, takie jak routing, uwierzytelnianie czy autoryzacja. Framework posiada również wbudowany kontener wstrzykiwania zależności (DI), który jest kluczowy dla budowy luźno powiązanych i łatwych w testowaniu systemów. Mechanizm ten, oparty na zasadzie odwrócenia sterowania (IoC), pozwolił na automatyczne dostarczanie instancji serwisów, takich jak `GeoLogDbContext`, do konstruktorów kontrolerów, co znacząco uprościło zarządzanie cyklem życia obiektów. Całość uzupełnia zaawansowany system routingu, który przy użyciu atrybutów (`[HttpGet]`, `[HttpPost]`) umożliwił intuicyjne mapowanie żądań na odpowiednie metody akcji w kontrolerach [22].

Entity Framework Core (EF Core): Jako warstwę dostępu do danych zastosowano maper obiektowo-relacyjny (ORM) Entity Framework Core. Technologia ta stanowi pomost pomiędzy obiektowym kodem aplikacji a relacyjnym modelem bazy danych, umożliwiając interakcję z danymi za pomocą zapytań w składni LINQ (Language-Integrated Query) zamiast pisania surowych zapytań SQL. EF Core automatycznie tłumaczy te operacje na zoptymalizowany dialekt SQL, co przyspiesza proces programowania i minimalizuje ryzyko

błędów, w tym podatności na ataki typu SQL Injection. Kluczowym mechanizmem wykorzystanym w projekcie jest śledzenie zmian (ang. **Change Tracking**), dzięki któremu kontekst bazy danych (*'DbContext'*) automatycznie generuje niezbędne instrukcje *'INSERT'*, *'UPDATE'* lub *'DELETE'* na podstawie modyfikacji dokonanych na obiektach w pamięci. Ponadto, wbudowane narzędzie migracji pozwoliło na wersjonowanie schematu bazy danych w podejściu **Code-First**, zapewniając spójność modelu danych w kodzie ze strukturą bazy na każdym etapie rozwoju aplikacji [23].

PostgreSQL i rozszerzenie PostGIS: Jako system zarządzania bazą danych (SZBD) wybrano PostgreSQL, który jest zaawansowanym, obiektowo-relacyjnym systemem typu open-source, znanym z wysokiej wydajności, niezawodności i ścisłej zgodności ze standardem SQL. Jego architektura pozwala na łatwe rozszerzanie funkcjonalności, co było kluczowe w kontekście wymagań projektu. Najważniejszym rozszerzeniem wykorzystanym w aplikacji jest PostGIS, które przekształca PostgreSQL w potężny serwer danych geoprzestrzennych, w pełni zgodny ze specyfikacją Simple Feature Access zdefiniowaną przez Open Geospatial Consortium (OGC). PostGIS wprowadza nie tylko dedykowane typy danych, takie jak *'GEOGRAPHY'* (do obliczeń na sferoidzie) i *'GEOMETRY'* (do obliczeń na płaszczyźnie), ale również bogaty zestaw zoptymalizowanych funkcji przestrzennych (np. do obliczania odległości, buforowania czy analizy przecięć). Co więcej, PostGIS umożliwia tworzenie specjalistycznych indeksów przestrzennych, takich jak GiST (Generalized Search Tree), które drastycznie przyspieszają wykonywanie zapytań filtrujących dane na podstawie ich położenia geograficznego, co jest fundamentalne dla wydajności aplikacji [13].

Biblioteka NetTopologySuite: W celu umożliwienia operacji na danych geoprzestrzennych w ramach aplikacji .NET, wykorzystano bibliotekę NetTopologySuite. Pełni ona rolę kluczowego pomostu pomiędzy obiektywnym kodem C# a standardami OGC, implementując w zarządzanym środowisku .NET hierarchię typów geometrycznych, takich jak *'Point'*, *'LineString'* czy *'Polygon'*. Jest to de facto standardowa biblioteka do pracy z danymi przestrzennymi w tym ekosystemie. Jej zastosowanie było niezbędne do tworzenia i manipulowania obiektami geometrii przed ich zapisaniem do bazy danych, a także do interpretowania danych odczytanych z PostGIS. Co istotne, dostawca bazy danych Entity Framework Core dla PostgreSQL (*'Npgsql.EntityFrameworkCore.PostgreSQL'*) posiada wbudowaną, natywną integrację z NetTopologySuite, co umożliwiło płynne i transparentne mapowanie typów geometrycznych z C# na typy PostGIS bez konieczności pisania ręcznych konwerterów. Dzięki temu zapytania LINQ mogły zawierać operacje przestrzenne, które EF Core potrafił przetłumaczyć na odpowiednie funkcje SQL w dialekcie PostGIS [24].

Biblioteka AutoMapper: W celu zachowania czystej architektury i ścisłej separacji pomiędzy modelem domeny (encjami bazodanowymi) a obiektami transferu danych (DTO), wykorzystano bibliotekę AutoMapper. W architekturach wielowarstwowych, takich jak zaimplementowana w projekcie, konieczność konwersji obiektów pomiędzy warstwami jest częstym i powtarzalnym zadaniem. Ręczne przepisywanie wartości z pól jednego obiektu do drugiego jest nie tylko czasochłonne, ale również podatne na błędy i trudne w utrzymaniu, zwłaszcza przy rozbudowanych modelach. AutoMapper rozwiązuje ten problem poprzez konwencję nad konfiguracją (ang. **convention-over-configuration**). Biblioteka automatycznie mapuje właściwości o tych samych nazwach, a jednocześnie pozwala na definiowanie niestandardowych, złożonych reguł transformacji dla bardziej skomplikowanych przypadków. W projekcie GeoLog pozwoliło to na zdefiniowanie profili mapowań w jednym miejscu, co znacząco zredukowało ilość powtarzalnego kodu w kontrolerach i zwiększyło czytelność logiki biznesowej [25]

3.1.2 Technologie warstwy klienckiej (frontend)

Interfejs użytkownika został zrealizowany, jako Aplikacja Jednostronicowa (SPA) z wykorzystaniem popularnych bibliotek ekosystemu JavaScript.

Jako główną technologię do budowy interfejsu użytkownika wybrano bibliotekę React. Jej popularność wynika z deklaratywnego podejścia do tworzenia UI oraz architektury opartej na komponentach. Pozwala to na dekompozycję złożonych interfejsów na małe, niezależne i reużywalne części. Kluczowym mechanizmem optymalizacyjnym Reacta jest wykorzystanie wirtualnego DOM (Virtual DOM), czyli reprezentacji rzeczywistej struktury dokumentu w pamięci. Dzięki niemu, zamiast dokonywać kosztownych manipulacji bezpośrednio na drzewie DOM przeglądarki, React oblicza minimalny zestaw zmian, a następnie efektywnie je aplikuje, co zapewnia wysoką wydajność renderowania, szczególnie w przypadku dynamicznych widoków, takich jak interaktywna mapa [11]. Filozofia Reacta celowo odchodzi od tradycyjnego paradygmatu separacji technologii (HTML, CSS, JavaScript) na rzecz separacji odpowiedzialności (ang. separation of concerns). Zamiast oddzielnych plików dla struktury, logiki i stylów, React promuje organizację kodu w oparciu o komponenty, które hermetyzują wszystkie te aspekty w jednym miejscu. Umożliwia to składnia JSX, która jest rozszerzeniem języka JavaScript pozwalającym na deklaratywne opisywanie struktury interfejsu przy użyciu znaczników przypominających HTML bezpośrednio w kodzie. Takie podejście nie tylko poprawia czytelność i łatwość utrzymania kodu, ale również otwiera zaawansowane możliwości, takie jak renderowanie po stronie serwera (SSR), co pozwala na tworzenie uniwersalnych aplikacji o zoptymalizowanym czasie pierwszego ładowania.[26]

Material-UI (MUI): W celu zapewnienia spójności wizualnej, responsywności oraz przyspieszenia procesu tworzenia interfejsu użytkownika, zdecydowano o zastosowaniu biblioteki komponentów Material-UI. Jest to rozbudowany, otwartoźródłowy zestaw gotowych do użycia komponentów dla React, który implementuje zasady i wytyczne systemu projektowania Material Design, opracowanego przez firmę Google [27]. Wybór tej biblioteki nie był podyktowany jedynie dostępnością szerokiej gamy predefiniowanych elementów (takich jak przyciski, pola formularzy, karty czy okna dialogowe), ale przede wszystkim zaawansowanymi mechanizmami, które standaryzują i upraszczają złożone aspekty tworzenia warstwy wizualnej. Kluczową zaletą MUI jest zaawansowany system stylizacji, domyślnie oparty na podejściu CSS-in-JS z wykorzystaniem biblioteki Emotion. Pozwala on na definiowanie stylów komponentów bezpośrednio w kodzie JavaScript, co umożliwia ich dynamiczne modyfikowanie w zależności od przekazanych właściwości (props) lub globalnego motywu aplikacji. Takie podejście, w przeciwieństwie do tradycyjnych, statycznych arkuszy CSS, ułatwia tworzenie w pełni hermetyzowanych i reużywalnych komponentów o zmiennym wyglądzie [28]. Ponadto, MUI dostarcza zestaw tzw. komponentów systemowych, które stanowią fundament do budowania złożonych i responsywnych układów strony. Zastosowanie biblioteki pozwoliło skrócić czas potrzebny na tworzenie warstwy wizualnej i poświęcenie większej ilości nakładu pracy na stworzenie logiki biznesowej.

Do implementacji interaktywnej mapy, która jest centralnym elementem wizualizacji tras, wykorzystano bibliotekę React Leaflet. Jest to popularny zestaw komponentów React, który stanowi warstwę abstrakcji nad potężną, otwartoźródłową biblioteką mapową Leaflet.js [29]. Wybór tej technologii był podyktowany jej deklaratywnym podejściem: zamiast imperatywnie manipulować obiektem mapy, React Leaflet pozwala na definiowanie warstw mapy (takich jak kafelki, polilinie, czy znaczniki), jako komponentów React. Taka architektura idealnie wpisuje się w model działania całej aplikacji, umożliwiając łatwą integrację stanu aplikacji (np. listy punktów trasy) z elementami wyświetlanymi na mapie.

W projekcie wykorzystano darmowe kafelki mapowe dostarczane przez projekt OpenStreetMap, które stanowią domyślne źródło danych dla biblioteki Leaflet.

Biblioteka Recharts: Do wizualizacji danych statystycznych, takich jak profil wysokościowy i wykres prędkości trasy, wykorzystano bibliotekę Recharts. Jest to kompozycyjna biblioteka do tworzenia wykresów, zbudowana w całości na komponentach React [30]. Jej kluczową zaletą jest deklaratywne podejście, które idealnie wpisuje się w architekturę aplikacji. Zamiast imperatywnie manipulować elementami DOM, wykres jest budowany poprzez komponowanie gotowych elementów, takich jak `<LineChart>`, `<XAxis>` czy `<Tooltip>`. Dane do wizualizacji są przekazywane, jako właściwość (prop), a biblioteka sama zarządza procesem renderowania. Taka architektura pozwoliła na ścisłą integrację wykresów ze stanem aplikacji i umożliwiła szybkie stworzenie interaktywnych wizualizacji przy zachowaniu czystości i czytelności kodu.

3.1.3 Środowisko pracy i narzędzia deweloperskie

Proces tworzenia oprogramowania był wspierany przez starannie dobrany zestaw narzędzi, który zapewnił efektywność pracy i spójność projektu na każdym etapie.

Do implementacji warstwy serwerowej wykorzystano zintegrowane środowisko programistyczne (IDE) Microsoft Visual Studio 2022. Jego wybór był podyktowany głęboką integracją z ekosystemem .NET, co przejawiało się w zaawansowanych narzędziach do debugowania kodu, profilowania wydajności oraz wbudowanym menedżerze pakietów NuGet. Menedżer ten umożliwił łatwe wyszukiwanie, instalowanie i zarządzanie zależnościami projektu, takimi jak biblioteki Entity Framework Core czy NetTopologySuite, co znacząco uprościło konfigurację środowiska [31].

Z kolei do tworzenia warstwy klienckiej (frontend) wykorzystano edytor kodu Visual Studio Code. Jest to lekkie, lecz potężne narzędzie, które dzięki rozbudowanemu systemowi wtyczek i rozszerzeń zapewnia doskonałe wsparcie dla nowoczesnych technologii webowych, w tym języka JavaScript i biblioteki React. Elastyczność konfiguracji oraz terminal zintegrowany z edytorem ułatwiły pracę z menedżerem pakietów npm oraz innymi narzędziami ekosystemu JavaScript [32].

Do zarządzania bazą danych PostgreSQL wykorzystano narzędzie administracyjne pgAdmin 4. Umożliwiło ono nie tylko tworzenie i modyfikację schematu bazy danych w sposób graficzny, ale również podgląd przechowywanych danych, w tym wizualizację obiektów geometrycznych zapisanych przy użyciu PostGIS. Było to nieocenione wsparcie w procesie debugowania operacji na danych geoprzestrzennych, pozwalając na szybką weryfikację poprawności zapisywanych współrzędnych [33].

Całość kodu źródłowego projektu była zarządzana przez system kontroli wersji Git. Jest on obecnie standardem w branży oprogramowania, umożliwiając precyzyjne śledzenie historii wszystkich zmian w plikach projektu. Zastosowanie Gita pozwoliło na tworzenie i przełączanie się między gałęziami deweloperskimi, bezpieczne eksperymentowanie z nowymi funkcjonalnościami oraz przechowywanie kopii zapasowej projektu w zdalnym repozytorium (w tym przypadku GitHub) [34].

Literatura

- [1] Strava Inc.: Oficjalna strona internetowa. Dostęp: <https://www.strava.com/>, 2025.
- [2] Komoot GmbH: Oficjalna strona internetowa. Dostęp: <https://www.komoot.com/>, 2025.
- [3] Garmin Ltd.: Garmin Connect - Oficjalna strona internetowa. Dostęp: <https://connect.garmin.com/>, 2025.
- [4] Wikipedia: Strava. Dostęp: <https://pl.wikipedia.org/wiki/Strava>, 2025.
- [5] Strava Inc.: Dołącz do milionów sportowców i poszukiwaczy przygód. Dostęp: <https://www.strava.com/onboarding>, 2025.
- [6] Wikipedia: Komoot. Dostęp: <https://en.wikipedia.org/wiki/Komoot>, 2025.
- [7] OpenStreetMap Wiki: Komoot. Dostęp: <https://wiki.openstreetmap.org/wiki/Komoot>, 2025.
- [8] Garmin Ltd.: Garmin Connect: All Your Health and Fitness Stats, All in One Place. Dostęp: <https://connect.garmin.com/>, 2025.
- [9] Bass L., Clements P., Kazman R.: Software Architecture in Practice. 3rd ed., Addison-Wesley Professional, 2012.
- [10] Fowler M.: Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.
- [11] Haber, C.: Single Page Application (SPA). W: "Learning React: Modern Patterns for Developing React Apps", 2nd ed., O'Reilly Media, 2020.
- [12] Fielding R. T.: Architectural Styles and the Design of Network-based Software Architectures. Rozdział 5: "Representational State Transfer (REST)". Dysertacja doktorska, University of California, Irvine, 2000.
- [13] Obe R. O., Hsu L. S.: PostGIS in Action. 3rd ed., Manning Publications, 2021.
- [14] Jones M., Bradley J., Sakimura N.: JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force (IETF), 2015.
- [15] Connolly T., Begg C.: Database Systems: A Practical Approach to Design, Implementation, and Management. 6th ed., Pearson, 2015.
- [16] Chen P.: The Entity-Relationship Model—Toward a Unified View of Data. ACM Transactions on Database Systems, Vol. 1, No. 1, 1976, str. 9-36.
- [17] PostgreSQL Global Development Group: PostgreSQL 16.3 Documentation. Dostęp: <https://www.postgresql.org/docs/16/index.html>, 2025.
- [18] Provos N., Mazières D.: A Future-Adaptable Password Scheme. W: "1999 USENIX Annual Technical Conference", 1999.
- [18] Provos N., Mazières D.: A Future-Adaptable Password Scheme. W: "1999 USENIX Annual Technical Conference", 1999.
- [19] Troelsen, A., Japikse, P.: Pro C# 12 with .NET 8. 12th ed., Apress, 2024.
- [20] Michaelis, M., Tour de Nuts, K.: Essential C# 12. Addison-Wesley Professional, 2024.
- [21] Albahari, J.: Concurrency in C# Cookbook. 2nd ed., O'Reilly Media, 2019.
- [22] Freeman A.: Pro ASP.NET Core 6. 10th ed., Apress, 2022.
- [23] Lerman, J., Miller, R.: Programming Entity Framework Core. O'Reilly Media, 2016.
- [24] NetTopologySuite Development Team: NetTopologySuite Documentation. Dostęp: <https://nettopologysuite.github.io/NetTopologySuite/>, 2025.
- [25] Bogard, J.: AutoMapper Documentation. Dostęp: <https://automapper.org/>, 2025.
- [26] Santana Roldan, C.: React 17. Wzorce projektowe i najlepsze praktyki. Projektowanie i rozwijanie nowoczesnych aplikacji. Helion, 2021.
- [27] Google: Material Design 3 Documentation. Dostęp: <https://m3.material.io/>, 2024.
- [28] Chan, M.: A Thorough Analysis of CSS-in-JS. W: "CSS-Tricks", 2022. Dostęp: <https://css-tricks.com/a-thorough-analysis-of-css-in-js/>, 2025.
- [29] Agafonkin, V.: Leaflet - an Open-Source JavaScript Library for Interactive Maps. Dokumentacja. Dostęp: <https://leafletjs.com/>, 2025.

- [30] Recharts Development Team: Recharts - A composable charting library built on React components. Dokumentacja. Dostęp: <https://recharts.org/>, 2025.
- [31] Microsoft: Visual Studio 2022 Documentation. Dostęp: <https://docs.microsoft.com/en-us/visualstudio/ide/>, 2025.
- [32] Microsoft: Visual Studio Code Documentation. Dostęp: <https://code.visualstudio.com/docs>, 2025.
- [33] The pgAdmin Development Team: pgAdmin 4 Documentation. Dostęp: <https://www.pgadmin.org/docs/pgadmin4/latest/index.html>, 2025.
- [34] Chacon, S., Straub, B.: Pro Git. 2nd ed., Apress, 2014.

Sygnatura:

POLITECHNIKA RZESZOWSKA im. I. Łukasiewicza
Wydział Elektrotechniki i Informatyki

Rzeszów, Rok

STRESZCZENIE (wybierz rodzaj pracy)

TYTUŁ PRACY

Autor: Imię Nazwisko, nr albumu: (wybierz symbol studiów)-123456

Opiekun: (tytuł naukowy przed) Imię Nazwisko(tytuł naukowy po)

Słowa kluczowe: (max. 5 słów kluczowych w 2 wierszach, oddzielanych przecinkami)

(tekst streszczenia - max. 10 wierszy)

RZESZOW UNIVERSITY OF TECHNOLOGY
Faculty of Electrical and Computer Engineering

Rzeszow, Rok

DIPLOMA THESIS (wybierz rodzaj pracy) ABSTRACT

TYTUŁ PRACY W WERSJI ANGIELSKIEJ

Author: Imię Nazwisko, code: (wybierz symbol studiów) -123456

Supervisor: (tytuł naukowy przed) Imię Nazwisko (tytuł naukowy po)

Key words: (max. 5 słów kluczowych w 2 wierszach, oddzielanych przecinkami)

(tekst streszczenia w jęz. angielskim - max. 10 wierszy)